From Park to Pitch: Applying (Un)Supervised Learning to MLB Analytics

Alex Yoon, Justin Tseng, Caroline Gaede
SIADS 696 | Spring 2026

Introduction

Modern Major League Baseball (MLB) operates at the intersection of physics, strategy, and data. With tools like Statcast, every pitch and batted ball is tracked with high-resolution measurements of velocity, spin, movement, location, and many other metrics. Despite this abundance of data, many questions in the field of baseball analytics remain largely unexplored. For example, how can we define and classify pitch types in a way that reflects actual behavior? What impact do ballpark characteristics have on gameplay? Are there noticeable patterns in injury types or frequencies? Answering questions such as these can inform coaching decisions and help players grow their skillset and understand shortcomings. This is why we have set out to apply both supervised and unsupervised machine learning techniques to see if the classification of batted balls or the clustering of pitch types can reveal actionable insights to us as stakeholders in the community of MLB analytics.

Our supervised approach to classifying batted balls was a multiclass process where possible outcomes were Single, Double, Triple, Homerun, and Out. Based on scalability and interpretability, we explored Random Forest models, a Gradient Boosting Classifier, XGBoost, LightGBM, LinearSVC, and Logistic Regression. Given the nature of our dataset's structure, hyperparameter optimization proved to be quite the challenge, although we ultimately overcame this obstacle and produced a highly accurate LGBM model. This final model was evaluated based on the F1 metric, and the LGBM model outperforms the other models we tested for nearly every class possible. Understanding the behavioral outcomes of batted balls is just the beginning in terms of quantifying offensive performance and understanding player skill within context, and this model could be used to generate a number of insights for players and coaches alike.

Our unsupervised approach challenges the traditional classification of pitch types by grip and investigates measurable movements and spin characteristics to uncover latent pitch groupings. For scalability, we explored multiple clustering algorithms such as k-means, agglomerative, and DBScan. Evaluating outcomes with an elbow plot and silhouette plot, we revealed that the optimal number of clusters is 5-15, with k-means and agglomerative performing relatively similarly on the dataset while DBScan is more unreliable. In other words, the distribution and structure of pitch types appears to be driven by features pulled from our dataset rather than by traditional methods, also suggesting that many seemingly distinct pitches can actually collapse into a smaller number of movement-based pitch archetypes. Solving this problem will extend to scouting, pitch design, and batter preparation or selection; overall this solution offers a more objective way to characterize pitch behavior.

Related Work

1. *A Machine Learning Model that Classifies Pitch Type Better Than I Can,* by Robbie Dudzinski
    a. This 2025 project builds a machine learning (ML) pipeline using publicly available data that spans from 2016-2025. Dudzinski omitted some pitches and re-grouped others, then used several different ML methods in a final pipeline to categorize those pitches based on the remaining pitch types selected. While this project has pre-determined pitch buckets for each pitch (excluding a few due to their relative unpredictability), our project differs in that we plan on clustering these pitches without pre-determined categories. More simply, we are tackling this from an unsupervised clustering perspective. This means that we may end up with more or less different pitch types than Dudzinski's work; further, we are using fewer seasons than in his work. This will allow the data to

speak for itself and for us to interpret its behavior freely rather than restricting pitch types to pre-determined classes.

2. *Fixing the Curve: Improving Major League Baseball Pitch Classification with Model-Based Clustering,* by Gina M. Assaf et al.
    a. Assaf applied Gaussian mixture models (GMM) and other clustering methods to pitch tracking data in an attempt to improve on the MLB's pitch labels by bringing in movement data and velocity metrics. This work demonstrates that this data-driven clustering approach can better align with true pitch behavior than many traditional grip-based classifications. While this work primarily focuses on model-based clustering, we choose to evaluate clustering structure across multiple algorithms (k-means, agglomerative, and DBScan). Therefore, we examine cross-algorithm consistency and conduct further sensitivity analysis to reveal which attributes most drive the formation of the clusters.
3. Bayesball: A Bayesian Hierarchical Model for Evaluating Fielding in Major League Baseball, by Shane T. Jensen, Kenneth E. Shirley, and Abraham J. Wyner
    a. Jensen, Shirley, and Wyner developed a probabilistic modeling framework to evaluate batted ball outcomes using movement metrics, ball location, and trajectory to estimate the likelihood of hits and defensive performance. This was some of the earlier work done on using high-resolution ball-tracking data to model probabilistic outcomes instead of relying on aggregate statistics. While this focuses primarily on defensive evaluation, we expand this concept into a full multiclass classification problem to predict discrete outcomes (Single, Double, Homerun, etc.). We also incorporate environmental and contextual variables and evaluate using macro-F1 to address class imbalance. Our project is a classic extension of the groundwork laid by this framework.
4. *On the Injured List: Analyses of Injuries Among MLB Players*, Milestone I Project by Caroline Gaede, Justin Tseng, and Ryan Erickson
    a. While our current project is not a direct extension of this previous injury analysis, a great amount of inspiration, motivation, and domain knowledge stemmed from two of our team members completing Milestone I. We explored injury risk and likelihood across players and teams to understand their frequency and severity in the recent MLB seasons. Again, we did not explore injuries in this analysis, but the primary motivation began with this initial project.

Data Sources & Preparation

We pulled Statcast data and park dimensions from pybaseball and Baseball Savant respectively. Pybaseball is a Python library that scrapes pitching and batting data from Statcast, Baseball Reference, Baseball Savant, and FanGraphs. After completing data ingestion, we were able to pull Statcast data for every ball pitched and batted for the 2021-2025 seasons. Initially, we had a high-dimensional dataframe containing each data point scraped from Statcast. Variables and columns of interest include *events*, *bat speed*, *launch speed*, *hc_x*, *hc_y*, and situational information such as *outs*, *balls*, and *strikes*. Some columns were discarded but used in the creation of calculated fields. For example, the *details* column was used to create the *batter name*, and *team name* was used to bring in the Baseball Savant park factor data. This secondary dataset did not have a direct export option, so we copied a spreadsheet for each season and were able to download CSV files from there. Each file contains information about distances from various field markers (home plate, left field, center, etc.), fence heights, deepest points, and more. This environmental data ended up being crucial in our comprehensive analysis.

The supervised portion of this analysis required further preparation before we could embark on feature selection. Since a type of hit is heavily dependent on where the ball lands on the field, knowing the angle at which the ball left the pitcher or bat matters significantly. To address this, we created a new feature called *spray angle* using Bill Petti's R code to convert Statcast's *hc_x* and *hc_y* values to an angle. The model can take the *hc_x* and *hc_y* values along

with the angle to understand the location of the ball along with other movement-related metrics. After this step, we now have the following complete list of features:

- **Numerical Variables:** *launch_speed, launch_angle, spray_angle, balls, strikes, outs_when_up, bat_speed, hc_x, hc_y, LF Line, LF Gap, CF, RF Gap, RF Line, Deepest Point, Playing Field Area (sq. ft), Avg. Fence Distance, Avg. Fence Height, altitude*

- **Binary Variables:** *on_1b, on_2b, on_3b*

- **Categorical Variables:** *stand, p_throws*

Additionally, before continuing to feature selection, we needed to address missing values for the numerical features using Scikit-Learn's SimpleImputer. Using the median for the strategy makes the most sense since launch speed and bat speed are both dependent on the physical abilities of each athlete - which is information we don't have. We then used StandardScaler to standardize different units to a common scale. Although it was not necessary for all models, this allows for more efficient model testing. To address missing categorial features, we used SimpleImputer again but with strategy set to constant rather than median, and set *fill_value* as "missing". We lastly used OneHotEncoder to convert strings to integers and filled in missing binary features with zero. At this point, we are prepared to embark on the actual parameter tuning and model selection portion of the analysis.

Part A: Supervised Learning

I. Methods Description

Our overarching goal was to predict the outcomes (Single, Double, Triple, Homerun, Out, etc.) of batted balls using supervised multiclass classification. Considering scalability, interpretability, and model diversity, we selected the following models for exploration: Random Forest, HistGradientBoostingClassifier, XGBoost, LightGBM, LinearSVC, and Logistic Regression. Let's address each model separately.

The Random Forest classifier builds multiple decision trees using bootstrapped samples of the data and a random subset of features. Each tree splits the data until the specified max depth parameter is met. The final prediction is determined by majority vote across all the trees. We selected this model because it reduces overfitting, handles non-linear relationships well, and performs effectively with many different feature types.

HistGradientBoostingClassifier is based on gradient boosting but incorporates histogram-based binning to significantly improve computational efficiency. The traditional gradient boosting was attempted during the initial stages of our project, but due to the size of our dataset, training was very slow. The histogram approach allowed us to scale efficiently while retaining the advantages of boosting.

The XGBoost builds decision trees sequentially, where each new tree attempts to correct the residual errors of the previous one. It includes built-in regularization to prevent overfitting and supports parallel processing for improved scalability. The final prediction is the sum of the corrections from all the trees. By default it uses the most conservative tree option, but we utilized histogram for efficiency.

Similar to XGBoost, LightGBM (LGBM) was chosen for speed and scalability. It also builds many small decision trees sequentially, but it grows trees leaf-wise using gradient based one side sampling (GOSS). By splitting the leaf with the largest loss first, LightGBM often achieves strong accuracy while maintaining its efficiency.

To achieve model diversity, we also explored classic logistic regression. It is a probabilistic linear classifier that uses the softmax function to estimate the class probabilities. It serves as a strong baseline model and provides interpretable coefficients that indicate feature importances.

LinearSVC is a margin-based classifier that finds the optimal separating plane by maximizing class separation. It focuses on creating the maximum margin between the classes. Because it is built on liblinear, it scales well with large datasets and supports different penalty and loss function options.

During the model evaluation, we observed poor performance in predicting home runs and triples. Upon examining the distribution of events in our dataset, we found that outs accounted for ~70% of the dataset, while homeruns and triples accounted for ~5%. This caused the models to favor the majority class with some models completely ignoring triples and homeruns. To address this, we balanced the dataset by ensuring that outs and hits accounted for the same percentage of the dataset. This adjustment kept the relative rarity of homeruns and triples while preventing the models from becoming too biased towards predicting outs. After rebalancing, both accuracy and f1-scores improved substantially across all models.

Hyperparameter optimization was another challenge due to the size of the dataset. With over 600,000 data points, using GridSearchCV would have required excessive computational power and time. Instead, we pivoted to RandomizedSearchCV with 40-50 iterations. We first used *train_test_split* with *test_size = 0.4* to create a smaller dataset with the same class distribution. RandomizedSearchCV samples parameter combinations from predefined search spaces, allowing us to explore hyperparameters without exhaustively evaluating every single possibility. Although this method doesn't guarantee the best possible parameter, it provides a good balance between computational time and model performance given our limited time.

For evaluation, we selected the f1-macro as our primary scoring method. Because of the class imbalance, overall accuracy alone would overemphasize the accuracy of the majority class. The f1- score assigns equal weight to each outcome, ensuring that the accuracy of rare events (triples, homeruns) are as important as common outcomes (outs, singles). Since we are predicting 5 possible outcomes, it is important that the models predict all 5 outcomes accurately.

II. Supervised Evaluation

As previously mentioned, we chose f1-macro-score as the metric since we can't simply rely on accuracy of the models to predict the majority class (out). To have a useful model, we need it to predict all five outcomes accurately - not just one. The f1-macro-score will place even weights on each of the outcomes, so having a high score indicates the model's ability to predict all 5 outcomes accurately.

model_name	accuracy_mean	accuracy_std	f1_macro_mean	f1_macro_std
Random Forest	0.820667	0.002905	0.637660	0.006578
LinearSVC	0.516917	0.001251	0.363371	0.001697
Logistic Reg.	0.464700	0.002328	0.373246	0.001499
Hist. Gradient Boosting	0.805083	0.001326	0.650488	0.004909

XGBoost	0.835067	0.002514	0.648720	0.004565
LightGBM	0.821867	0.002328	0.651409	0.005175

Figure 1: Crossvalidation Table (with 60k train size)

	Random Forest	**SVM**	**Logistic Regression**	**HistGBM**	**XGBoost**	**LGBM**
Out	0.945469	0.751476	0.652093	0.900195	0.944289	0.949984
Single	0.865433	0.352691	0.376016	0.777084	0.843862	0.886144
Double	0.870553	0.160714	0.195246	0.576676	0.761269	0.877074
Triple	0.903141	0.036741	0.036401	0.172286	0.250316	0.961003
Home Run	0.944068	0.411180	0.414513	0.861673	0.926664	0.969649
accuracy	0.922463	0.583207	0.490404	0.831085	0.908526	0.931576
macro avg	0.905733	0.342560	0.334854	0.657583	0.745280	0.928771
weighted avg	0.923061	0.608663	0.549528	-	0.906621	0.932455

Figure 2: Table of f1-scores for each model across all possible outcomes; LGBM maintains the highest f1-scores across all outcomes

Feature	**XGBoost**	**LGBM**
num__launch_angle	0.234237	105889
num__hc_y	0.216989	132461
num__hc_x	0.123791	109403
num__launch_speed	0.085252	146603
num__spray_angle	0.080268	95634
cat__stand_R	0.054133	11629
bin__on_1b	0.017369	9661
bin__on_2b	0.015641	6717
bin__on_3b	0.014872	4312
num__bat_speed	0.014712	62049
cat__p_throws_R	0.014342	8772
num__LF Gap	0.013120	25924
num__LF Line	0.013114	27105
num__RF Gap	0.013029	24620

num__Avg. Fence Distance	0.013026	597
num__Playing Field Area (sq. ft.)	0.013014	45306
num__Deepest Point	0.012931	21537
num__altitude	0.012727	24956
num__CF	0.012667	23268
num__Avg. Fence Height	0.012387	41074
num__RF Line	0.012380	24983

Figure 3: Weights per feature for XGBoost and LGBM models

III. Supervised Trade-Offs & Failure Analysis

Evaluating the feature sensitivity of the LGBM model, we found that the model's predictive power jumps significantly from top 2 to top 4 features. Above that, the model shows minimal gains (see Figure 4). Most of the predictive power is concentrated in the top 4 features, meaning that the model probably only needs those features to predict the outcome of the batted balls accurately. Thus, the other 17 features just add unnecessary complexity and increased training time without adding much to the accuracy.

Top k = _ features	f1-macro score
2	0.374456
4	0.632027
8	0.636895
21	0.639342

Training Size	f1-macro score
10k	0.649098
50k	0.700686
100k	0.749454
300k	0.882705

Figure 4: Sensitivity Report for LGBM; feature sensitivity analysis was conducted on 10k samples

The same thing cannot be said about the training size. With more training data, the model shows significant improvements in f1-score. The training time, as well as computational cost, also increases with training data size, but given the increased score, it is well worth the time to train on more data. We haven't reached diminishing returns with the training data size, but given that our model peaked at about 0.9 f1-score with ~400k data points, the diminishing return is close to the size of this dataset. With the sensitivity testing, we used the same parameter but smaller sample size of 10k due to time constraints.

Just like any other ML models, our models were not always successful, and investigating the failures can reveal just as much information as any one success. Looking at one instance where the model predicted *Single* instead of *Out*, we found that the *predict_prob* function gave 40% *Out* and 57% *Single*. Looking at the *lime_tabular* (Figure 5), we see that *hc_y* feature gave high predictive power for out, but *launch angle, hc_x, spray_angle, launch_speed*, and *bat_speed* supported *Single*. We could have fixed this by doing better feature engineering. *Launch_angle, bat_speed,* and *hc_y* should all be correlated, but in this case they are not. Our model doesn't factor the defender's ability to

field the ball. Even if the ball is hit well, the defenders can make great plays and prevent hits. If we could incorporate that as a feature, we may have been able to fix the issue.

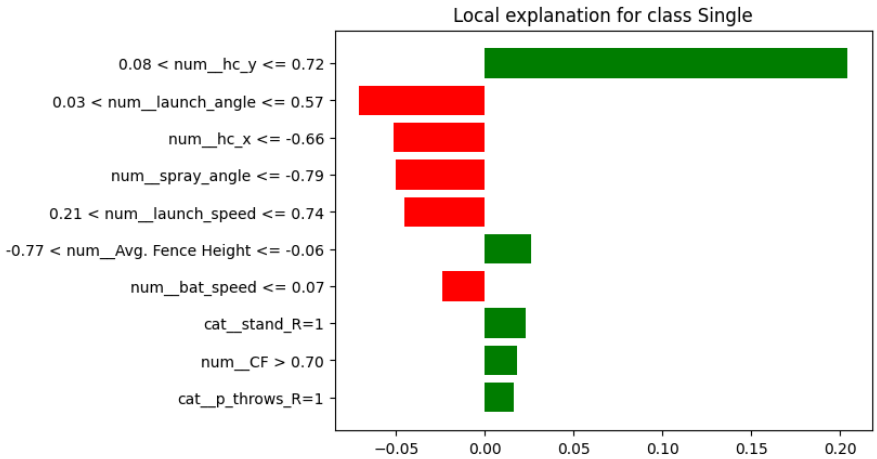

Figure 5: Local explanation for a misclassification

In terms of the model complexity, we chose *num_leaves*, since LGBM differs from other models in that it grows trees leaf-wise. Using the *validation_curve* method from sklearn, we cross-validated 1, 16, 32, 64, 128, 256 leaves. From our optimization process, the optimal number of leaves was 128. After running the validation curve, we probably could have used 64. This is where RandomizedSearchCV likely failed since it doesn't conduct exhaustive search for the best parameter. We could have better optimized the parameters by reducing the number of parameters to start. Most likely, some parameters aren't as impactful as other ones, so reducing the number of searches would have likely reduced computational time.

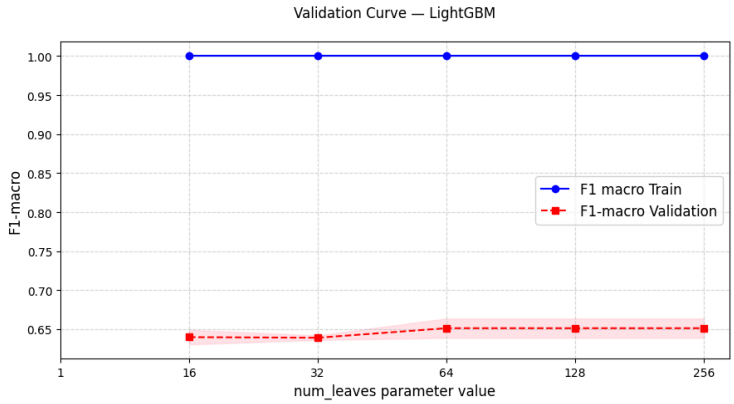

Figure 6: Validation Curve to identify optimal number of leaves

Finally, we used SHAP to see features' contribution to each prediction. We analyzed the top 4 features, since other features added unnecessary complexity and clutter to the visualization. In the previous model performance, we saw that *Double* had the lowest accuracy. Looking at the SHAP (Figure 7), we see that with 100 instances of failed prediction for *Double*, low values of *hc_y* and *launch_angle* were driving the model away from the correct prediction while high values of *hc_x* were driving the model away from the correct prediction. This indicates that the model associates these features more strongly with other outcomes. *Doubles* and *Triples* are a combination of ball placement and baserunning; since the runners have to sprint as fast as possible without being tagged out, placement alone can't predict doubles. If we added batter's sprint speed to the feature, we may have mitigated the issue.

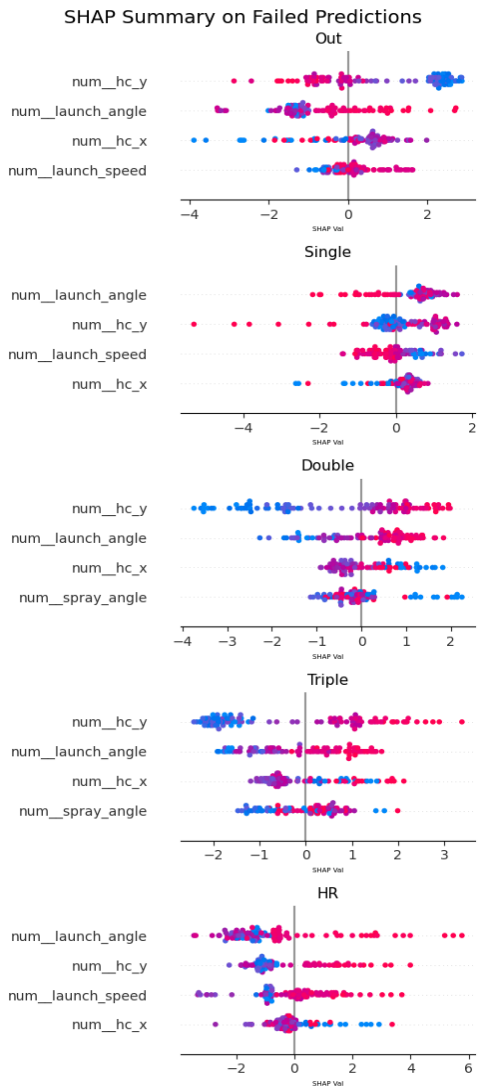

Figure 7: SHAP Summary on failed predictions

Part B: Unsupervised Learning

I. Methods Description

For the unsupervised learning portion of our project, we wanted to explore pitch types in baseball. In order to induce strikeouts and other negative outcomes for the batter, pitchers will use various pitch speeds and spin rates to change where the pitch will arrive to the batter. Traditionally, pitches are classified based on how the pitcher holds the ball and how they release it as they deliver it to the plate. However, different pitchers using the same pitch grip can achieve different locational outcomes from each other. To better classify these pitches, we wanted to cluster pitches based on data from Baseball Savant (instead of using the traditional method of pitch grips).

Our data set contains a high number of features, but only some are relevant to pitching. The features selected were as follows (descriptions from Baseball Savant):

1. *pfx_x*: Horizontal movement in feet from the catcher's perspective.
2. *pfx_z*: Vertical movement in feet from the catcher's perspective.
3. *release_speed*: Pitch velocities recorded as they leave the pitcher's hand.
4. *release_spin_rate*: Spin rate of pitch tracked by Statcast.
5. *spin_axis*: The Spin Axis in the 2D X-Z plane in degrees from 0 to 360, such that 180 represents a pure backspin fastball and 0 degrees represents a pure topspin (12-6) curveball.

To create a baseline expectation for what the data would show, we completed a preliminary analysis of the features selected. We observed where missing values appear, modeled rough distributions for variables (Figure 8), created a correlation matrix (Figure 9) and looked for outliers.

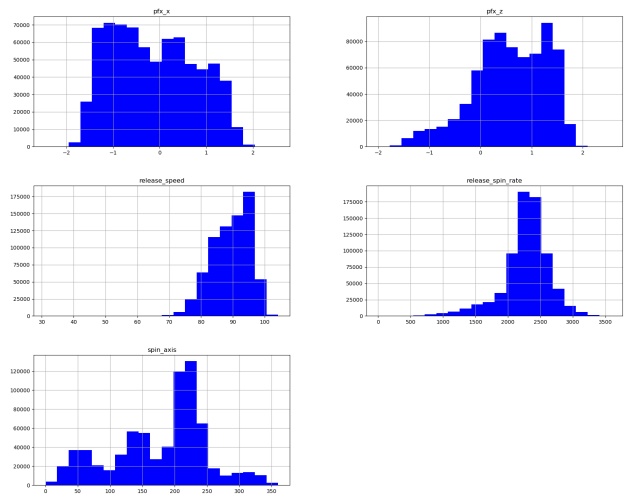

Figure 8: Distributions for top 5 features

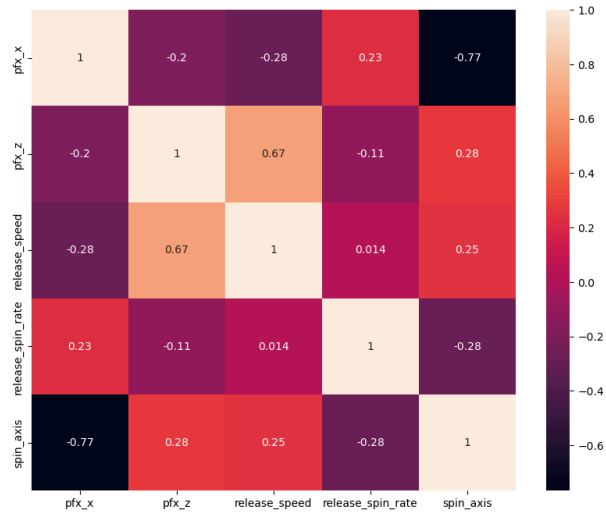

Figure 9: Correlation matrix for top 5 features

II. Unsupervised Evaluation

For the unsupervised learning workflow, we initially opted to use three clustering algorithms: KMeans clustering, agglomerative clustering, and DBScan. We opted for clustering algorithms, as our project's goal was to try to group pitch types separately from pitcher grips. Other forms of unsupervised learning didn't make sense for our project, as our data wasn't multi-dimensional, and our primary goal was to find specific connections or groupings between data points. All three methods were selected due to their scalability with large and very large data sets (in our 2025 dataset alone, there were 740,000 recorded pitches).

To determine the optimal amount of clusters, we created an elbow plot (Figure 10), a silhouette score plot (Figure 11), and a dendrogram (Figure 12). The elbow method helps determine the optimal number of clusters in a dataset and silhouette score measures cluster quality by calculating how similar a data point is to its own cluster compared to other clusters from -1 to 1. After reviewing the plots, we determined that the optimal number of clusters was 5, with a significant dropoff in efficacy beyond 15 clusters (as seen in the silhouette score plot). After testing cluster sizes ranging from 2 to 25, the maximum silhouette score was achieved using 5 clusters with the score of 0.347. While there are many more pitches than just 5, many of the pitches are a variation of other pitches. For example, 2 seam is a variation of 4 seam and sweeper, slider, and cutter share the same general movement but with varying degrees.

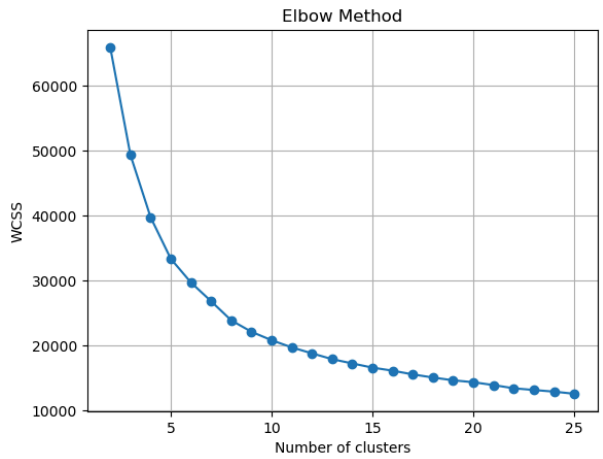

Figure 10: Elbow plot for optimal cluster amount

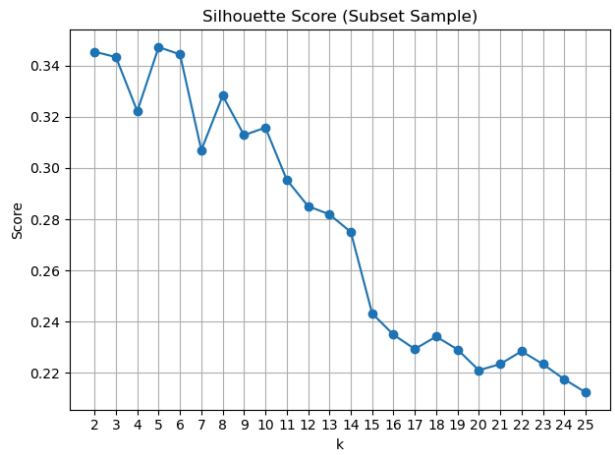

Figure 11: Silhouette score plot for cluster quality

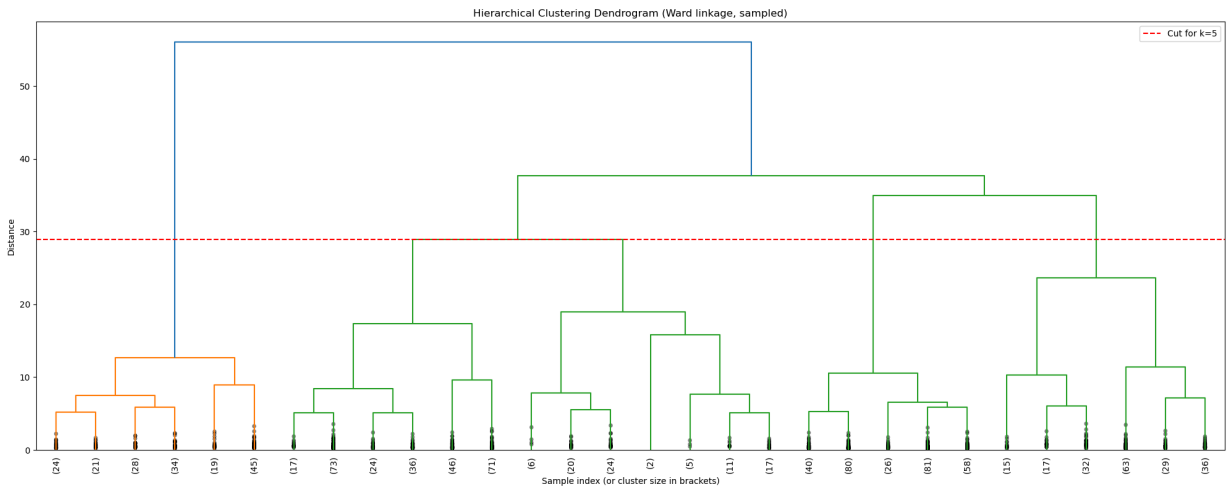

Figure 12: Dendrogram using Ward linkage

We also explored using DBScan for both optimal cluster counts and overall clustering, but the algorithm identified the vast majority of data points as the same cluster (Figure 13a), either as an actual cluster or noise. This result didn't make sense with either our other clustering methods or the original pitch grips from each pitch (which had 18 different types). Because of this, we did not utilize the results from DBScan in our final analysis. This conclusion is also further supported by an ARI comparison (as part of our sensitivity analysis), shown in Figure 13b as well. Adjusted Rand Index (ARI) is an evaluation metric to measure the similarity between different clusterings of the same dataset. As seen in the diagram below, KMeans and Agglomerative grouped the same dataset relatively similarly. But DBScan's score of 0 indicated its clustering matching the other models was essentially no better than random chance.

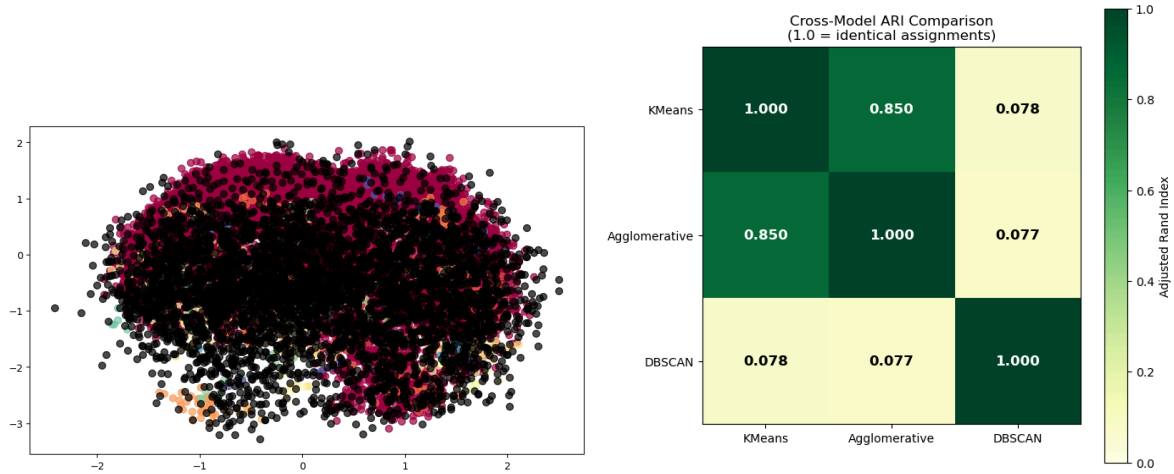

Figure 13a & Figure 13b: Results from DBScan

Overall, the clustering results[1] appear to be consistent between both clustering algorithms (K-Means and agglomerative). The approximate locations of the cluster centroids were also in agreement between the models. With a smaller sample size, the distribution of the movements of each pitch are easier to visualize compared to the full dataset.

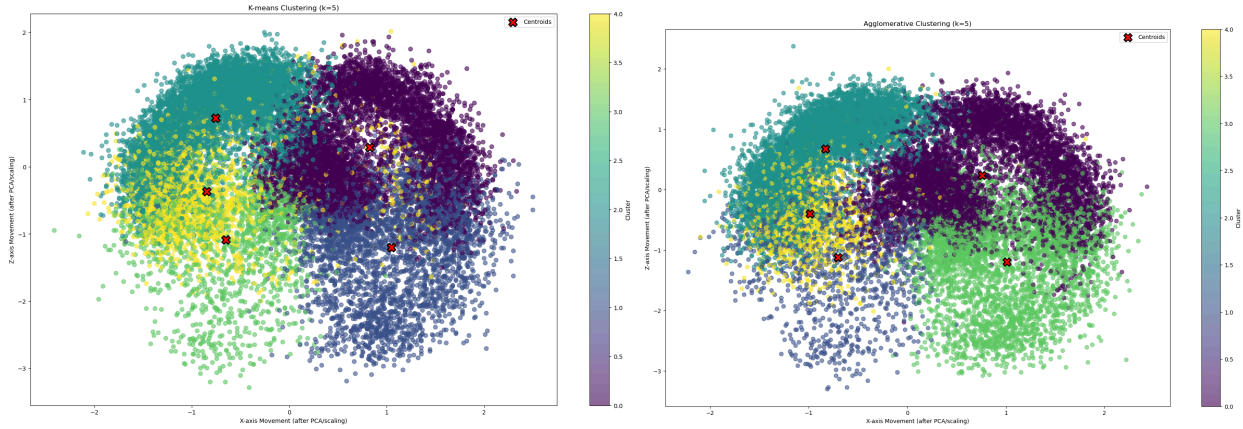

Figure 14: Results of KMeans and Agglomerative Clustering

III. Unsupervised Validation & Failure Analysis

Model validation consisted of an ARI comparison as well as a cluster size distribution (Figure 15). Given the close correlation between KMeans and agglomerative clustering in overall correlation as well as the relatively close sizes in cluster size distributions, the clustering is roughly consistent between the two selected clustering algorithms. The distribution of clustering provides a general idea of pitch distribution in MLB. Fastball type pitches are probably the most frequent because it is less straining on the elbows and they are pin point accurate. Whereas breaking balls (balls with high movements horizontally and vertically) are not as common due to strain on the elbows.

[1] Due to computing constraints, all clustering was performed using a randomized subset of the full data set.

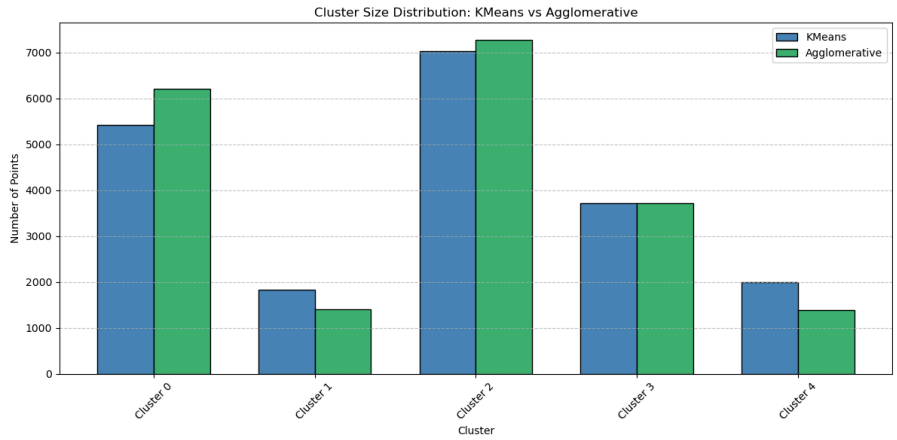

Figure 15: Distribution of cluster size for KMeans and Agglomerative clustering

For sensitivity analysis, we performed feature sensitivity analysis by removing one feature at a time from the dataset and recalculating the ARI values to determine the overall effect of each feature on the score. Both models were relatively close to each other through sensitivity analysis, but the *pfx_x* feature had the greatest impact on ARI vs. the baseline value of 1.0 (Figure 16).

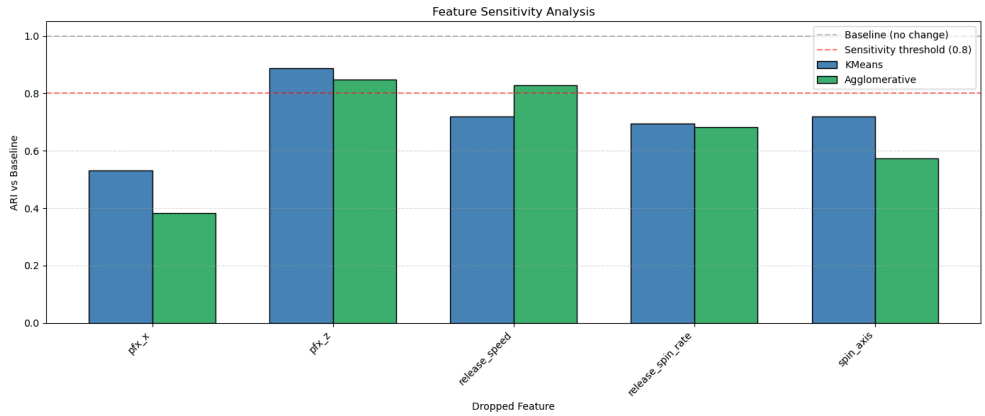

Figure 16: Feature sensitivity analysis revealing *pfx_x* feature with greatest impact on ARI

After evaluating the clustering results, we came to several conclusions.

1. As a whole, the clusters seem to be strongly correlated with the final pitch location. This is corroborated by the sensitivity analysis showing the heavy emphasis on the *pfx_x* feature (although interestingly, the *pfx_z* feature does not show this emphasis). The focus is on the final pitch location rather than how the ball is delivered *(release_speed, release_spin_rate,* and *spin_axis*) potentially imply that a greater focus should be placed on the end location of the pitch, and less what kind of grip the pitcher uses to deliver it.
2. In general, pitches appear to be skewed in the negative Z direction. This trend may be caused by strategic decisions. The lower a pitch is, the more likely it is to contact the ground, which makes it more difficult for the catcher to block. As there is a strong negative outcome associated with unsuccessful blocking (wild pitches or passed balls that allow a runner to advance), pitchers are likely encouraged to avoid pitches that are too low.

Beyond the unsupervised portion we explored, we could perform further analyses. Given the time for next steps, we could have split clustering between left- and right-handed players. In baseball, there is a large focus on left- and right-handed players, as both pitchers and batters gain an advantage relative to the other (but not at the same time) when considering handedness of the pitcher and the batter. Further analysis of the data when considering handedness could reveal new trends in the clustering that might result in a different number of clusters for each kind of handedness.

Additionally, further exploration of the DBScan algorithm to determine why it produced significantly different results. DBScan produced a much higher amount of clusters than anticipated, and was significantly different from both KMeans and agglomerative clustering (as shown in the ARI comparison matrix). Future efforts should explore the root causes of this discrepancy, and determine what needs to be adjusted to ensure DBScan will work on the data.

Takeaways & Next Steps

Overall, we learned quite a bit from pitch type clustering. We discovered that the selection of features is very important, and what features are included or excluded has a significant impact on the final clustering results (as can be seen in the sensitivity analysis). We also learned that scaling our features was important. All of our features are scales of magnitude apart - spin rate easily reaches 3000 RPM, while vertical movement maxes out around 3 feet. If we didn't scale our features, the results would have been very unrepresentative of the results we should've expected to see.

One surprise from the unsupervised learning portion of the project was the optimal number of five clusters. In the original dataset, Baseball Savant identified 18 different pitch types, but our analysis revealed a much lower optimal cluster number. It brings up a philosophical question: Is it worthwhile classifying different pitches based on delivery if they ultimately result in the same final outcome? Furthermore, if two pitchers throw the same pitch with the same pitch grip but the pitches end up in drastically different locations, should they be considered the same pitch?

Another interesting insight was that the most important variables aren't necessarily what kind of pitch and how hard it was thrown. Instead, there should be more importance on how the ball travels from the pitcher's hand to the batter, as well as the spin direction and rate induced on the ball by the pitcher. Almost every feature analyzed in our clustering project showed more sensitivity to the model than pitch release speed. This is most likely due to spin affecting the movement of the pitch.

The main challenge we encountered during unsupervised clustering was the results of the DBScan analysis. As seen in our results, the clustering results created by the DBScan were significantly different from the KMeans and agglomerative clustering algorithm results. Despite attempts to tune the DBScan object, almost all of our data points were being marked as the same (either in the same cluster, or as noise). We believe that the cause of this was because our data had a relatively even spread, with no clear groupings of points (especially noticeable in plots of the full dataset).

With more time, our solution could be extended into other seasons beyond the selected range. We could also run our analysis on either a larger subset (or the whole subset) of our original dataset, or extend the data into other seasons beyond our initial scope. Another exploration would be the connection between our clusters and the pitch grips described by the original dataset. Ultimately, pitch types as reported in both the dataset and in live baseball broadcasts are educated guesses based on what the pitches a player purports to throw, as well as what path the pitch takes to the batter. This is further emphasized with the tactical advantage gained in deception during pitch delivery

(as batters can catch on to differences in pitches based on a pitcher's body movement during their pitch). With further exploration, we could better classify these pitches and potentially help develop strategic advantages for teams or others interested in pitcher performance.

Finally, combining the findings from our project and applying them to other relevant baseball statistics could help define true expected outcomes from a pitcher's given pitch. For example, if a batter struggles against pitches high and inside, it might be more worthwhile to go to a pitcher who can consistently pitch effectively to that position, rather than selecting a pitcher based on the pitch grip he uses that usually results in the correct kind of movement to reach the position. Conversely, batters could better prepare against pitchers if they know how a given pitch is supposed to travel to the plate based on what specific cluster a pitcher's pitch falls into, and use that information to create a more positive outcome from their at bat.

Ethical Considerations

I. **Supervised Ethical Considerations**
   - **Fairness & Representation Bias:** Rebalancing the dataset did improve f1-macro performance but simultaneously altered the distribution of outcomes which may distort the real-world probabilities. This could accidentally overemphasize rare events (Triples, Home Runs, etc.) and misrepresent the true player performance. Compare balanced and unbalanced datasets to observe differences.
   - **Player Evaluation & Economic Impact:** If used in scouting or broadcasting analytics, systemic misclassification could disadvantage certain player archetypes. For example, some models may bias evaluation toward easily quantifiable skills. Be sure to vet models for unintentional biases.
   - **Transparency & Overreliance:** Although SHAP and LIME help improve interpretability, boosted tree models remain complex and may encourage automation bias if stakeholders put too much trust in the predictions. Clear communication here is necessary so results are not misused.

II. **Unsupervised Ethical Considerations**
   - **Player Evaluation & Labeling Risk:** Clustering pitches based on movement metrics rather than traditional grips could redefine pitcher categorization and evaluation. If used in scouting, these clusters could mislabel pitches and influence player reputation or contract value without real player input.
   - **Feature Selection & Structural Bias:** Since we placed a heavy emphasis on *pfx_x* for clustering, this could over-prioritize pitch location instead of the mechanics of the pitch itself. This could oversimplify pitching strategy and involve structural bias by privileging easily measurable outcomes.

Statement of Work

All group members contributed equally to this project. Alex Yoon primarily contributed to the supervised learning portion of the project, and Justin Tseng primarily contributed to the unsupervised learning of the project. Caroline Gaede contributed to both portions of the project, and primarily contributed to final report composition.

References

Assaf et al., *Fixing the Curve: Improving Major League Baseball Pitch Classification with Model-Based Clustering*. 2018, Causeweb USA Proceedings, https://www.causeweb.org/usproc/sites/default/files/usresp/2018-2/Fixing%20the%20Curve%20Improving%20Major%20League%20Baseball%20Pitch%20Classification%20with%20Model-Based%20Clustering.pdf.

Dudzinski, Robbie. "A Machine Learning Model that Classifies Pitch Type Better Than I Can." *Medium*, 6 Oct. 2025, https://medium.com/codex/a-machine-learning-model-that-classifies-pitch-type-better-than-i-can-8691ec18d190.

Gaede, Tseng, Erickson, *On the Injured List: Analyses of Injuries Among MLB Players*, 2025, Google Slides, https://docs.google.com/presentation/d/1rES4rImXLrLyCvtw9Uf4YQaa3MMzJkYg/edit

Jensen, Shane T., Kenneth E. Shirley, and Abraham J. Wyner. *Bayesball: A Bayesian Hierarchical Model for Evaluating Fielding in Major League Baseball*. 2009.

Petti, Bill. "Research Notebook: New Format for Statcast Data Export at Baseball Savant." *The Hardball Times,* 28 Apr. 2017, https://tht.fangraphs.com/research-notebook-new-format-for-statcast-data-export-at-baseball-savant/.